

Cost based Random Forest Classifier for Intrusion Detection System in Internet of Things

K. Pramilarani^{a,*}, P. Vasanthi Kumari^b

^a Department of Computer Science and Engineering, Dayananda Sagar University, Bangalore, India

^b Department of Computer Applications, Dayananda Sagar University, Bangalore, India

HIGHLIGHTS

- Effective IDS is developed by proposing an Improved Random Forest Classifier.
- Feature importance based cost matrix is derived for improving the classification.
- The Spark, Kafka and Scikit-learn libraries of Python are used to improve the IDS.
- Maximum variance margin is used to remove the noise values from the input.

ARTICLE INFO

Keywords:

Cost matrix
Cost based Random Forest Classifier
Internet of Things
Intrusion Detection System
Kafka
Scikit-learn
Spark

ABSTRACT

Internet of Things (IoT) is the collection of physical and digital devices that are interconnected using Internet for exchange of information and delivery of services. The Internet of Things (IoT) is an extended application of Internet that is used to offer various services for users in the fields of agriculture, healthcare, education, smart homes and so on in the modern world. The significant issues of the intrusion present in IoT are network disconnection, network hacking and data theft from the source. So the challenging task for worldwide utilization of IoT is to address their security issues, because of the feature imbalance in the different types of attacks. The most essential task for addressing security issues is to predict and classify the intrusion in the network. In this paper, the Cost based Random Forest Classifier (CRFC) is proposed for developing an effective Intrusion Detection System (IDS). The CRFC based classification is improvised by incorporating the cost matrix calculated based on feature importance that helps to improve the process of splitting the features even if there is a feature imbalance. Further, three important libraries of Python namely, Spark, Kafka, and Scikit-learn are used in this IDS to improve the classification performances. In that, Spark is used to implement the distributed environment, Kafka is used for streaming the data and Scikit is used to implement CRFC. There are two datasets known as NSL-KDD and UNSW-NB15 that are used to evaluate the performance of the proposed CRFC-IDS method. The CRFC-IDS method is analyzed on the basis of accuracy, precision, recall, F1-Measure, Area Under the Curve (AUC), False Acceptance Rate (FAR) and Matthews Correlation Coefficient (MCC). The existing approaches OCSVM and DBF are used for comparison with the CRFC-IDS method. The accuracy of CRFC-IDS for NSL-KDD dataset is found to be 99.957%, which is highest when compared to OCSVM and DBF.

1. Introduction

The Internet of Things (IoT) integrates different smart devices that lead to the development of a large-scale network [1,2]. The tiny internet-linked smart devices embedded with sensors and actuators perform as facilitators for an effective connection with users [3]. IoT devices generate a huge amount of data that demand a large storage

space and computing resources [4]. Gartner computed that the usage of IoT would reach 26 billion units in 2020, while Statista's recent study discloses that its usage will be increased to 75.44 billion by 2025 throughout the world [5]. IoT systems are advantageous for various applications in the fields of environmental monitoring, home automation, smart cities, agriculture and farming, logistics and supply chain management, healthcare, digital twins design, public safety, smart

* Corresponding author.

E-mail addresses: Pramiselva2020@gmail.com (K. Pramilarani), vasanthi-bca@dsu.edu.in (P. Vasanthi Kumari).

<https://doi.org/10.1016/j.asoc.2023.111125>

Received 6 May 2023; Received in revised form 9 November 2023; Accepted 27 November 2023

Available online 5 December 2023

1568-4946/© 2023 Elsevier B.V. All rights reserved.

energy grids, quality of life for citizens and so on [6,7]. The intruders attack IoT devices as these devices transfer a huge amount of data with external resources because of inadequate security and defense. Thereby, a large amount of property loss is caused when IoT is affected by attacks. Therefore, the development of proper IDS which focuses on ensuring the security of IoT is crucial [8,9].

An IDS is utilized for securing the device's communication process, as well as detecting any intrusion in the network. The IDS observes and transmits the alarm to the system administrator when a malicious attack is identified in the network [10,11]. The main objective of this research is to identify the intrusions existing in the network using machine learning classifier. In recent times, the machine learning classifier is commonly used for performing the detection of intrusion [12]. Machine learning is a famous data-driven approach which is efficient in approaching and resolving engineering problems, specifically those that are extremely complex and nonlinear [13]. Since the effectiveness of conventional IDS is affected because of a large amount of data flow over the network, it is required to solve and handle the cyber-attacks with a huge amount of data flow. The three solutions that are used to handle the high data traffic in the network are; developing dedicated hardware, using big data approaches and using feature selection approaches. In particular, the big data approach named Spark is effectively used to handle large-scale network data traffic [14,15]. Generally, spark is considered one of the big data processing tools which used as a core component in each big data architecture [16]. Spark examines the datasets in a parallel manner using many nodes, and prioritizes speed [17]. The problem statement of this work is specified as follows: The higher false alarm rate and high traffic load are considered as important issues in the IDS, because it leads to cause the misclassification while detecting the intrusions. A high false alarm ratio means that the IDS is generating a large number of false positives, incorrectly classifying normal traffic as intrusions. A high traffic load makes it more challenging for the IDS to identify intrusions, as legitimate network activity may blend with potential malicious behavior. This leads to a decrease in recall because some true intrusions may be missed during the classification.

The contributions are listed as follows:

- CRFC is used to perform effective classification among the normal traffic and attack classes according to the cost matrix that is computed based on feature importance. Further, the attack classes are also classified for performing precise identification so as to secure the IoT network from threats. The CRFC automatically adapts to different attacks such as DoS, Probe, R2L and U2R based on the data and attack distribution. Hence, the CRFC's tree structures are generated based on the new adaptation to various attacks, providing the reliable intrusion detection information.
- Three important python libraries such as Spark for distributed environment implementation, Kafka for data streaming and Scikit-learn for CRFC design, are included.

The remaining paper is sorted as follows: Section 2 provides information about existing researches available on IDS. A detailed explanation of CRFC based IDS is provided in Section 3, and the results are presented in Section 4 along with the comparative analysis. Further, Sections 5 and 6 provides the discussion and conclusion.

2. Related work

The related work about existing researches available on IDS are given in this section along with their advantages and limitations.

Alazzam et al. [18] developed Intelligent Lightweight IDS which obtained a low false alarm rate while achieving a higher detection rate. The developed Network IDS (NIDS) was an integration of two key subsystems which operated in parallel, where each subsystem was trained by using the One-Class Support Vector Machine (OCSVM). Here, one

system was trained through normal packets and the other one was trained using attack packets. Further, the outcomes from both subsystems were integrated for offering the best prediction for each packet transmitted via the network. However, the higher false alarm affected the performance of anomaly detection subsystem using OCSVM.

Sarhan et al. [19] analyzed the efficiency of three different Deep Learning (DL) approaches used to predict the attack vectors with three Shallow Learning (SL) approaches. The considered DL models were deep feed forward, Convolution Neural Network (CNN) and recurrent neural network whereas the SL models were decision trees, Logistic Regression (LR) and naive Bayes. Further, three different Feature Extraction (FE) approaches such as auto-encoder, principal component analysis, and linear discriminant analysis were explored along with the classification models. From these different classifier and FE combinations, no single integration was the best in all the benchmarks which were ToN-IoT, CSE-CIC-IDS2018 and UNSW-NB15.

Alkadi et al. [20] developed Deep Blockchain Framework (DBF) for providing security based on distributed discovery of intrusion and privacy-based blockchain for IoT networks. The bidirectional long short-term memory was used to predict cyberattacks from the data migration in the network. Smart contracts and blockchains were used to suggest the privacy-preserving approach deployed to enable immutable data transfer and movement among multi-cloud services. But, DBF's performances were affected when traffic overload occurred because of heavy network traffic.

Alferaidi et al. [21] presented the distributed integrated deep learning IDS for the Internet of Vehicles according to the Apache Spark framework. The features and data were obtained on the basis of the combination of deep-learning CNN and improved Long Short-Term Memory (LSTM) using the cluster. The developed deep CNN-LSTM was used to detect the intrusion through the recognition of abnormal behavior and large-scale car network data traffic. However, the detection capacities of classifiers were required to be considered for further improving the prediction.

Baniasadi et al. [22] generated a training algorithm namely Neighborhood Search-Based Particle Swarm Optimization (NSBPSO) for tuning the parameters of deep architecture. The exploration and exploitation of particle swarm optimization were improved by using the NSBPSO algorithm. The merits of NSBPSO were used for optimal training of deep architecture to achieve the best accuracy. However, a huge amount of epochs were required to achieve the highest accuracy while detecting the intrusion.

Zeeshan et al. [23] presented the architecture of Protocol Based Deep Intrusion Detection (PB-DID) to identify intrusions in IoT. In PB-DID, a data packets was generated from IoT traffic based on the evaluation of features of Bot-IoT and UNSWNB15 datasets. This new dataset creation depended on the transmission control protocol and flow. The issues of over-fitting and imbalanced datasets were avoided in PB-DID during classification. However, this model was required to consider more attack classes for an effective analysis.

Bouya-Moko et al. [24] developed optimization-based deep learning to detect intrusions using Spark. The Fuzzy Local Information and Bhattacharya-based C-Means (FLIBCM) was used to divide the data, while class-wise information was used to choose the imperative features. Next, oversampling was used to augment the data and perform further. The Deep Maxout Network (DMN) was used to conduct the classification using augmented data in which Student Psychology Water Cycle Caviar (SPWCC) was used to choose the appropriate weights of DMN. The classification using DMN was evaluated only for individual classes, however, the overall analysis needed to be done among normal and attack classes to evaluate the efficiency.

Devprasad et al. [25] developed lightweight IDS based on the context adaptive classification through the ranking approach of Technique for Order of Preference by Similarity to Ideal Solution (TOPSIS). In this IDS, the first level statistical feature selection was done using chi-square, in combination with bat algorithm that was used to choose more relevant

features. Further, the ensemble voting classifier was used as per the choice selected by TOPSIS. The classification performances were affected because of the less amount of features selected during feature selection.

Mohamed and Ejebali [26] presented deep State-Action-Reward-State-Action (SARSA), a reinforcement learning for classifying the imbalanced and modern traffic network. SARSA was integrated with Deep Neural Network (DNN) for interacting with the input data where traffic was examined for detecting attacks. Specifically, the deep SARSA agents with automated trial-error policy were used to predict the attacks. The process of learning was stable due to the usage of on policy strategy in deep SARSA agents. However, the SARSA-DNN performed numerous computations because of its complex architecture.

Alqahtani, A.S [27] developed Firefly-Swarm-Optimized LSTM (FSO-LSTM) to predict the various types of attack along with increased amount of data entries. The spatial and temporal correlated features of IoT network and the optimized LSTM was utilized for predicting the attacks in IoT. The firefly algorithm used in the LSTM was used to minimize computational overhead and it was improved the prediction accuracy. The developed FSO-LSTM was required to feature importance for further enhancing the prediction.

Wang, S et al. [28] presented the IDS using ResNet, Transformer and spatial Bi-directional LSTM (BiLSTM) namely Res-TranBiLSTM which considered both the temporal and spatial features of network traffic. The data imbalance degree was avoided by using a Synthetic Minor Overriding Technique (SMOTE)-Edited Nearest Neighbor (ENN) approach. The temporal features of network traffic were preserved by using a BiLSTM. The Res-TranBiLSTM was provided less prediction accuracy for normal class, hence it denoted a higher false alarm rate because of concealment of the original features.

Therefore, the problems found in the related works are the degradation in performances due to higher false alarm rates and high traffic load. Some researchers considered only a less number of attack classes during the analysis. To overcome these, the CRFC-based classification is developed for achieving the precise classification between the normal traffic and attack classes. Here, efficient Python libraries such as Spark, Kafka, and Scikit are used with CRFC for further improving the classification.

3. CRFC-IDS method

In this research, effective intrusion detection is developed using CRFC with Spark, Kafka and Scikit libraries. The key processes of the CRFC-IDS method are 1) Dataset acquisition, 2) Preprocessing and 3) Classification. The cost matrix incorporated in the CRFC is used to avoid misclassification considering the feature importance. The splitting criteria of CRFC is enhanced based on the computed cost matrix which helps achieve reliable prediction even when it is processed with imbalanced features. Therefore, precise classification among normal traffic and attacks is achieved by using the CRFC-IDS method. Fig. 1 shows the block diagram of the CRFC-IDS method.

3.1. Data acquisition

For study purposes, two different datasets namely, NSL-KDD [29] and UNSW-NB15 [30] are considered for evaluating IDS. The NSL-KDD dataset is extended from KDD 99 [31] by resolving the intrinsic issues in KDD 99 such as data imbalance and redundancy. The labels of the NSL-KDD are generally divided as Normal and Attack, while the Attack is additionally divided into four categories which are U2R, probe, DoS and R2L. Here, the data types are similar to KDD 99, but the dissemination of data is different. Moreover, the NSL-KDD dataset has ~120,000 labeled samples. On the other hand, the UNSW-NB15 is created by IXIA PerfectStorm tool. The UNSW-NB15 has high amount of modern attack types (i.e., 9 types) and more user behaviors than the KDD 99 and NSLKDD. Therefore, the UNSW-NB15 totally has 257,673 records from

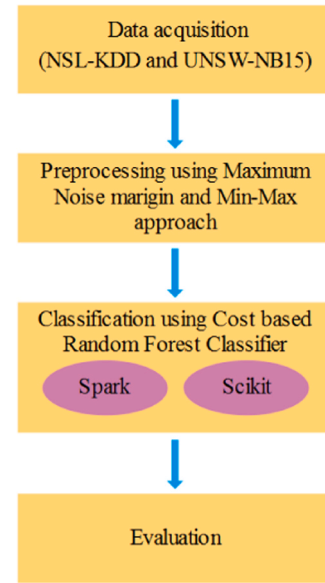


Fig. 1. Block diagram of CRFC-IDS method.

normal and attack traffic. UNSW-NB15's features are separated into five classes: basic, flow, content, time and additional generated features. The relationship between the normal traffic and attack types [32] for both the NSL-KDD and UNSW-NB15 is shown in Table 1.

3.2. Preprocessing

Data preprocessing suitably converts the input data to optimize data acquisition and the further processes. The preprocessing approaches considered in this IDS are the maximum variance margin and min-max algorithm. In maximum variance margin, the variance for all the data samples in the stream is calculated, then the mean value is considered as threshold. The samples within the threshold are streamed for further process and the remaining are determined as noise values. The determined noise values are eliminated from the stream of data. After noise removal, the data are further preprocessed by the min-max algorithm. In general, there is a huge difference between the minimum and maximum values of the dataset, therefore data normalization is used for reducing the algorithm's complexity and its relevant processes. The normalization operation depends on data scaling that has the min-max algorithm which can transform the data's current range to bring between the range $[-1, 1]$ and $[0, 1]$. Eq. (1) expresses the min-max algorithm.

$$x = \frac{(D - D_{\min})(\max - \min)}{(D_{\max} - D_{\min}) + \min} \quad (1)$$

Where D is input; $(\min, \max)$ is a specified range of input; an initial range of input is $(D_{\min}, D_{\max})$ and preprocessed output is x . The preprocessed data is given as input to CRFC to perform an effective classification.

3.3. Cost based Random Forest Classifier

The preprocessed data is given as input to the CRFC for classification purposes. The conventional RFC [33,34] is modified as CRFC by

Table 1
Attack and normal traffic proportions of the dataset.

Dataset	Type	Percentage (%)
NSL-KDD	Normal traffic	55.06
	Attack	44.94
UNSW-NB15	Normal traffic	53.56
	Attack	46.54

considering the cost function for avoiding misclassifications. The CRFC has the capacity for eliminating the high variance or bias during the identification and it is easy to design in an effective and precise way. The CRFC is used to maintain accuracy, even if a huge amount of data is missing during the analysis. The emergence of more trees i.e., Forests is used to improve the accuracy of CRFC. Each tree that exists in the forest provides the classification (i.e., vote) while predicting a new observation.

The classification of CRFC is chosen depending on a huge amount of votes from all the trees of the forest. Each fundamental tree is divided over only a random subset of available independent variables from bootstrapped sample data for creating the CRFC. For instance, the training data for each tree is randomly chosen with replacement. A forest with dissimilar trees is generated, because each tree utilizes distinct independent variables and training data. The bagging modification is used to build and average the huge amount of trees for achieving approximately unbiased models to reduce the variance. The schematic diagram of CRFC is shown in Fig. 2. The algorithm to develop the random forest with N amount of trees is denoted below:

For every $n = 1, 2, \dots, N$

- Use Bootstrap to develop the sample X_n .
- Use sample X_n to build the decision tree b_n .
- Based on the criteria, select the best attribute and perform the split in the tree until the sample data is exhausted.
- Tree is constructed until there are no higher than $n_{\min}$ objects at each leaf, or until tree achieves a significant height.
- From n initial ones, choose m amount of random features for each part to discover the best partitioning between the trees.

The final regression algorithm $f(x)$ is as expressed in Eq. (2).

$$f(x) = \frac{1}{N} \sum_{i=1}^N b_i(x) \quad (2)$$

Where, the regression tree is denoted as $b_i(x)$, and N defines amount of trees. The suggested amount of random features at regression tasks is $m = n/3$, where n defines the number of initial features. The correlation between the trees and the overfitting issue is avoided by using the random subspace method.

In CRFC, the Gini index condition based on the cost matrix is used for constructing the trees that are used to compute the condition of splitting according to the node's impurity. The impact of misclassification error from the cost used in the Gini value is used to provide equal level Gini value during the prediction that helps to eliminate misclassification. The CRFC chooses an appropriate splitting variable by making each node as pure as possible. Consider a portion of data reaching the g th node is A_g and amount of data points belonging to the class C_i is denoted as A_g^i . Therefore, the Gini index of node g (GI_g) is expressed in Eq. (3).

$$GI_g = 1 - \left(\sum_{i=1}^c (p_g^i)^2 \times \text{Cost}_g^i \right) \quad (3)$$

Where, p_g^i is relative frequency of i th class that is expressed in Eq. (4) and the cost matrix used for avoiding the misclassification is denoted as Cost which is expressed in Eq. (5).

$$p_g^i = \frac{A_g^i}{A_g} \quad (4)$$

The smaller value of Gini index of node g denotes higher purity which implies that the node has higher observations from a single class i . Therefore, the decrement in the Gini index is essential while splitting the node.

Various levels of consequences occur because of the various misclassifications. For example, the misclassification of R2L as Normal is more dangerous than the misclassification of DoS as Normal. The cost value computed for CRFC to avoid misclassification is expressed in Eq. (5). Further, the regression value ($C_{ij}(x)$) is computed using the Eqs. (6)

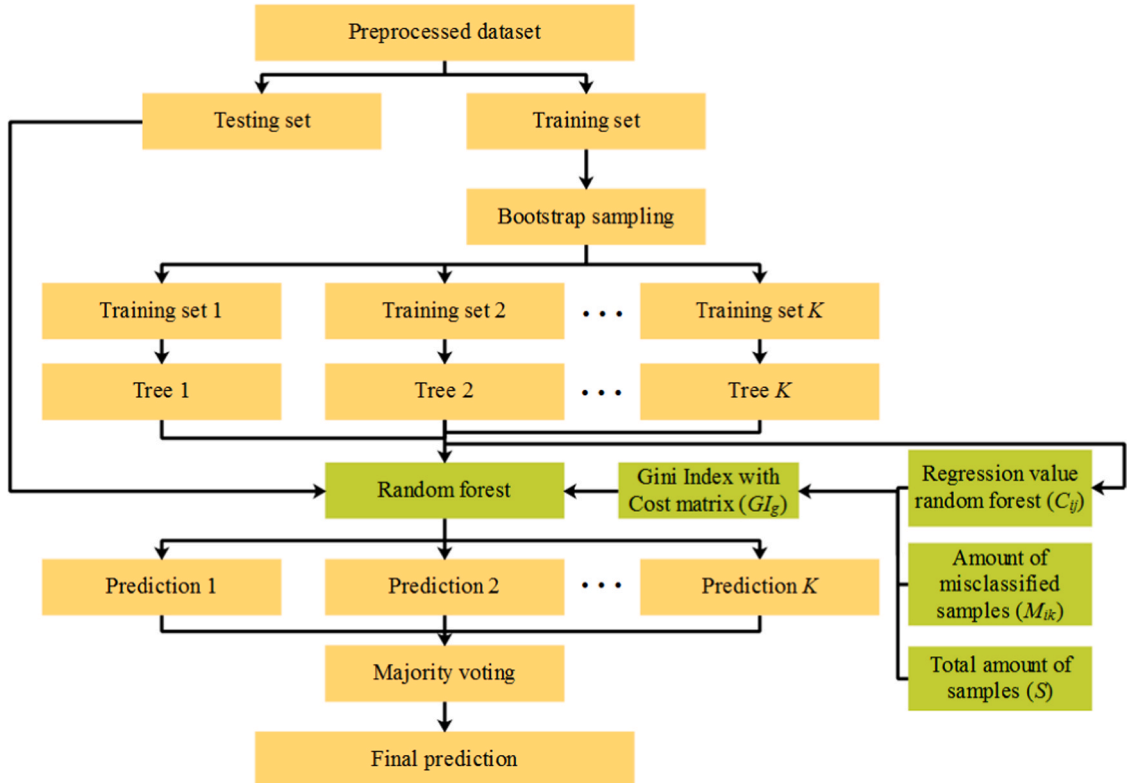


Fig. 2. Schematic diagram of CRFC.

and (7). The incorporation of cost regression is to eliminate the error caused by misclassification.

$$Cost = \frac{M_{ik} \times C_{ij}(x)}{S}, i, k = 1, 2, \dots, N \text{ and } j = 1, 2, \dots, F \quad (5)$$

$$C_{ij}(x) = \begin{bmatrix} f(x_{(0,0)}) & f(x_{(0,1)}) & \dots & f(x_{(0,i)}) \\ f(x_{(1,0)}) & f(x_{(1,1)}) & \dots & f(x_{(1,i)}) \\ \vdots & \vdots & \ddots & \vdots \\ f(x_{(i,0)}) & f(x_{(i,1)}) & \dots & f(x_{(i,i)}) \end{bmatrix} \quad (6)$$

Eq. (6) is further modified to identify a smallest minimum value as expressed in Eq. (7).

$$C_{ij}(x) = \operatorname{argmin}_{1 \leq (i,j) \leq N} \{C_{ij}(x)\} \quad (7)$$

Where, amount of samples in i th class misclassified as samples of k th class is denoted as M_{ik} ; F denotes the number of features; regression value is denoted as $C_{ij}(x)$, and the total amount of samples is denoted as S .

In that, the cost value is computed based on the importance of features from CRFC. The derived cost value is used to enhance the splitting of features even as the CRFC processes with less amount of data. The computed cost value denotes the features weightage that indicates the importance for a specific target or specific attack type. Subsequently, this cost matrix is used as weightage value for enhancing the condition of splitting criteria even when the data has imbalanced features. Therefore, the developed CRFC predicts the intrusion not only based on the probability of trained data, but also depends on the feature importance which helps to enhance the IDS. The developed CRFC automatically adapts to different types of attacks such as DoS, Probe, R2L and U2R because it depends on both the data and attack distribution. This kind of adaption is used to define the tree structures of CRFC that helps to provide the reliable intrusion detection. So, there is no probability of random decision due to the improvisation of splitting criteria based on the cost matrix.

3.3.1. Scikit-learn

Scikit-learn [35] is used to implement the proposed CRFC for performing classification. Scikit-learn is generally an open-source Python library that offers support to design machine learning approaches. This Scikit-learn depends on the SciPy and NumPy which are Python libraries for performing the scientific calculation. Scikit-learn has become famous because of the extensive usage of Python, and ML is used to feature the Scikit-learn. An effective operation over computing resources is achieved by optimizing an ML approach. Scikit-learn has excessive support with quality assurance, bug tracking and documentation. Additionally, the demonstration of input/output data is uniform in Scikit-learn.

3.3.2. Spark and Kafka

Spark [16] is used to implement the distributed environment to perform the CRFC for ensuring an effective IDS. Kafka is used to perform data streaming from the distributed environment of Spark. Spark is considered a big data processing tool which is one of the key components of each architecture of big data. The data saved in the repository or data consumed systems are analyzed by Spark. Apache Spark was developed by AMPLab at UC Berkeley in 2009, and it is suitable for an extensive usage on many cases. The common libraries developed at Spark processing engine are MLlib, Spark Streaming, and Spark SQL for machine learning and GraphX to perform computations of the graph. The supported programming languages of Spark are Scala, Python, R and Java. The cluster over Hadoop YARN and Mesos/ Kafka can operate Spark. Spark supports both batch and stream processing.

4. Results and discussion

This section shows the performance analysis of the CRFC-IDS

method. The implementation and execution of the CRFC-IDS method are done using Python 3.6 software. The system is configured with 1 TB memory Windows 10 operating system, 128 GB RAM and 22 GB RAM. There are two different datasets NSL-KDD and UNSW-NB15 considered for evaluating the CRFC-IDS method. The parameters such as accuracy, precision, recall, F1-Measure, AUC, FAR and MCC are expressed in Eqs. (8)–(14).

$$Accuracy = \frac{TP + TN}{TN + TP + FN + FP} \times 100 \quad (8)$$

$$Precision = \frac{TP}{TP + FP} \times 100 \quad (9)$$

$$Recall = \frac{TP}{TP + FN} \times 100 \quad (10)$$

$$F1 - measure = \frac{2Precision \times Recall}{Precision + Recall} \times 100 \quad (11)$$

$$AUC = \frac{1}{2} \times \left(\frac{TP}{TP + FN} + \frac{TN}{TN + FP} \right) \times 100 \quad (12)$$

$$FAR = \frac{FN}{TP + FN} \quad (13)$$

$$MCC = \frac{TP \times TN - FP \times FN}{\sqrt{(TN + FN)(TP + FP)(TN + FP)(TP + FN)}} \times 100 \quad (14)$$

Where, TP, TN, FP , and FN denotes true positive, true negative, false positive, and false negative, respectively.

4.1. Performance analysis for the NSL-KDD dataset

The performance of the CRFC-IDS method for the NSL-KDD dataset is analysed with different classifiers namely, Logistic Regression, Stochastic Gradient Decent (SGD), and Gradient Boosting Decision Tree (GBDT). Further, the performances are analysed in two different cases such as the classification of each attack type and the classification between attack and normal traffic. Tables 2 and 3 show the class-wise performance analysis of CRFC-IDS method for NSL-KDD dataset. Further, the graphical illustration of class-wise accuracies is shown in Fig. 3. This result shows that the classification performances of CRFC-IDS are improved in comparison with the Logistic Regression, SGD, and GBDT. The CRFC improves the classification based on the computation of cost value considering the feature importance. The computed cost value of CRFC is used to avoid misclassification between the classes. Specifically, the cost value enhances the splitting criteria of the features

Table 2

Class-wise performance analysis of CRFC for NSL-KDD by precision, recall and F1-Measure.

Classifiers	Class	Precision (%)	Recall (%)	F1-Measure (%)
Logistic Regression	DoS	98.8664	84.0817	91.3141
	Probe	89.9055	93.6726	90.6301
	R2L	90.6992	66.2278	68.4704
	U2R	90.0077	73.6049	79.3200
SGD	DoS	99.3025	85.3808	90.8089
	Probe	89.7688	93.8999	92.7259
	R2L	91.7424	66.9790	69.5014
	U2R	91.0981	74.7199	79.0912
GBDT	DoS	98.0988	85.2485	92.1446
	Probe	89.5955	94.0874	91.9322
	R2L	91.4651	66.8666	69.5316
	U2R	90.3520	74.3020	79.3280
CRFC	DoS	99.7295	85.1171	92.2279
	Probe	90.1443	94.6827	92.2915
	R2L	89.7700	66.2348	68.3121
	U2R	91.7707	73.8606	80.2394

Table 3

Class-wise performance analysis of CRFC for NSL-KDD by accuracy, FAR, AUC and MCC.

Classifiers	Class	Accuracy (%)	FAR (%)	AUC (%)	MCC
Logistic Regression	DoS	93.0557	0.2254	98.2369	0.7051
	Probe	93.4866	0.1738	98.2809	0.6194
	R2L	84.4864	0.3047	68.2858	0.0207
	U2R	99.1295	3.0488	97.8991	0.4132
SGD	DoS	93.0325	0.2296	97.5794	0.7117
	Probe	93.5729	0.1853	97.5820	0.6285
	R2L	84.6192	0.3076	68.2521	0.0277
	U2R	99.5559	3.0500	98.8614	0.4184
GBDT	DoS	92.8570	0.2375	97.9307	0.7149
	Probe	93.8985	0.1947	98.4016	0.6317
	R2L	84.4894	0.3149	68.2058	0.0323
	U2R	99.2658	3.0567	98.8267	0.4317
CRFC	DoS	92.7601	0.2408	97.8937	0.7207
	Probe	94.0715	0.2034	97.7318	0.6440
	R2L	84.0152	0.3219	68.4624	0.0350
	U2R	99.5184	3.0606	98.2772	0.4344

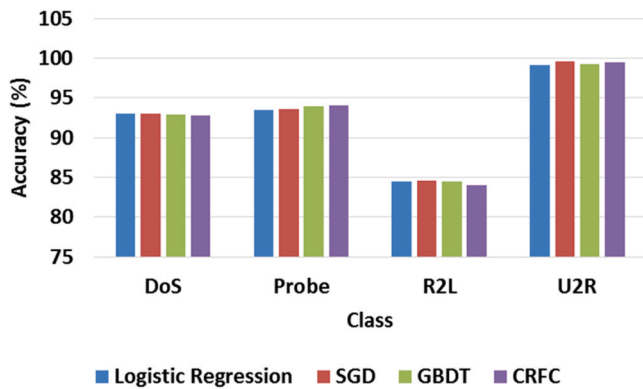


Fig. 3. Accuracy performances for class-wise prediction of NSL-KDD for different classifiers.

that are used to achieve reliable intrusion detection between different attacks such as DoS, Probe, R2L and U2R.

Tables 4 and 5 show the overall analysis of the CRFC-IDS method for the NSL-KDD dataset with different classifiers. The performances are analysed between the normal traffic and attack classes. All the attack types such as DoS, Probe, R2L and U2R are collectively considered as attack class. The accuracy comparison for CRFC-IDS with different classifiers is shown in Fig. 4. The developed CRFC-IDS achieves higher classification than Logistic Regression, SGD and GBDT. The average classification accuracy of CRFC is found to be 99.957%, whereas the Logistic Regression obtains an accuracy of 84.2589%, while the SGD obtains 98.1028% and the GBDT obtains 98.4662%. The misclassification among the normal traffic and attack classes is avoided by computing the cost value based on feature importance in the CRFC.

4.2. Performance analysis for UNSW-NB15 dataset

The performances of CRFC for UNSW-NB15 dataset are analyzed with different classifiers such as Logistic Regression, SGD, and GBDT. Tables 6 and 7 show the analysis of CRFC with different classifiers for

Table 4

The overall analysis of CRFC for NSL-KDD by precision, recall and F1-Measure.

Classifiers	Precision (%)	Recall (%)	F1 Measure (%)
Logistic Regression	89.7872	83.4399	86.2769
SGD	98.6980	98.9920	98.6721
GBDT	99.2897	98.5352	99.2434
CRFC	98.7589	98.6801	98.8110

Table 5

Overall analysis of CRFC for NSL-KDD by accuracy, FAR, AUC and MCC.

Classifiers	Accuracy (%)	FAR (%)	AUC (%)	MCC (%)
Logistic Regression	84.2589	0.3138	92.7424	0.5751
SGD	98.1028	0.5334	98.4267	0.3296
GBDT	98.4662	0.2792	99.8913	0.7217
CRFC	99.957	0.2519	100.00	0.2674

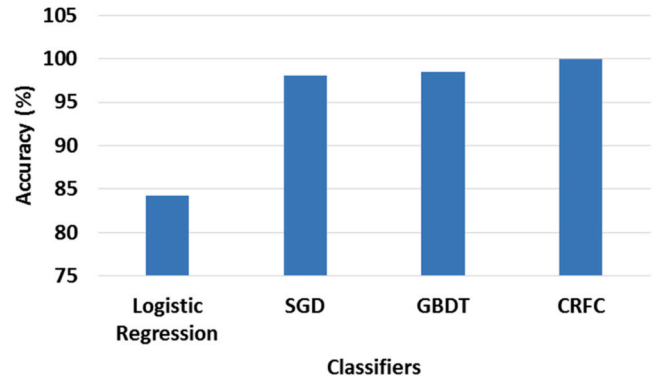


Fig. 4. Average classification accuracy of NSL-KDD dataset for different classifiers.

Table 6

Analysis of CRFC for UNSW-NB15 by precision, recall and F1-Measure.

Classifiers	Precision (%)	Recall (%)	F1 Measure (%)
Logistic Regression	91.7872	88.4399	90.2769
SGD	98.898	99.292	98.8721
GBDT	99.4897	98.7352	99.5434
CRFC	98.9589	98.9801	99.311

Table 7

Analysis of CRFC for UNSW-NB15 by accuracy, FAR, AUC and MCC.

Classifiers	Accuracy (%)	FAR (%)	AUC (%)	MCC (%)
Logistic Regression	92.4506	0.9578	90.4880	0.5326
SGD	98.8348	1.2062	94.5561	0.1889
GBDT	99.6037	0.3478	99.7907	0.3164
CRFC	99.953	0.2232	99.8151	0.2766

UNSW-NB15 dataset. Further, the accuracy comparison of CRFC with the UNSW-NB15 dataset is shown in Fig. 5. This analysis shows that the CRFC outperforms the Logistic Regression, SGD and GBDT. The CRFC exhibits better classification based on the incorporated cost value. The

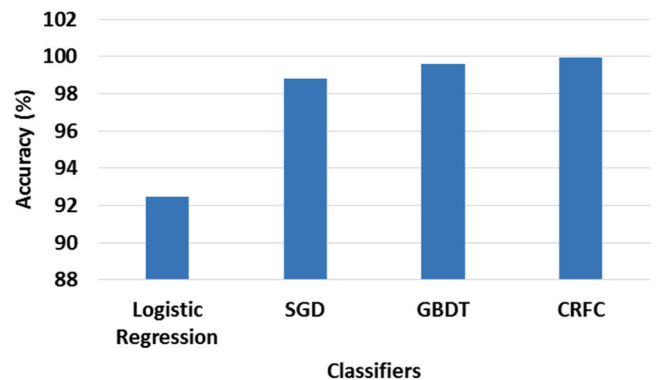


Fig. 5. Average classification accuracy of UNSW-NB15 dataset for different classifiers.

classification using CRFC is enhanced by using both the probability of trained data and enhanced splitting criteria taking into consideration the feature importance.

4.3. Comparative analysis

The comparative analysis for CRFC-IDS with existing research is shown in this section. Existing research approaches such as OCSVM [18] and DBF [20] are used to evaluate the CRFC-IDS method. Two datasets, NSL-KDD and UNSW-NB15 are considered for comparison. Tables 8 and 9 show the comparison of NSL-KDD and UNSW-NB15 dataset respectively. The accuracy comparison for CRFC-IDS is shown in Fig. 6. This comparison shows that the developed CRFC-IDS method outperforms the OCSVM [18] and DBF [20] for both datasets. The developed CRFC-IDS method overcome the issues created by higher false alarm ratio and high traffic load. CRFC minimizes false alarms in IDS by assigning higher misclassification costs to false positives, and it adapts to high traffic loads by adjusting Gini index condition based on cost matrix, making a balance between alerting and reducing false alarms. The false alarm ratio and traffic load is significantly impact on the accuracy and F1-measure of an IDS. For example, the accuracy of CRFC-IDS for the UNSW-NB15 dataset is 99.953%, whereas the OCSVM [18] obtains 99.3% and DBF [20] obtains 99.41%. The cost matrix formulation based on feature importance, is incorporated in CRFC for avoiding the misclassification among the attack types and normal classes. Therefore, the classification performances of CRFC-IDS are further increased by using the Spark, Kafka and Scikit-learn libraries.

5. Discussion

This section provides the brief discussion about the results achieved from CRFC-IDS developed for IoT. At first, the results of CRFC is evaluated with different classifiers including Logistic Regression, SGD and GBDT. The performance of CRFC-IDS are analyzed by using two different datasets NSL-KDD and UNSW-NB15. Specifically, the class wise performance is evaluated for the NSL-KDD dataset. From this analysis, it is cleared that the CRFC achieves better performance when compared to the logistic regression, SGD and GBDT. Moreover, the CRFC-IDS is compared with the OCSVM [18] and DBF [20] in the section of comparative analysis. The proposed CRFC-IDS achieves better performance than the OCSVM [18] and DBF [20] for both the datasets. For example, the accuracy of CRFC-IDS for NSL-KDD dataset is 99.957%, whereas the OCSVM [18] obtains 99.8%. The CRFC provides better IDS by incorporating the cost matrix to create the trees which are used to compute the condition of splitting according to the node's impurity which helps to avoid the misclassification during the analysis.

6. Conclusion

In the recent decades, a huge amount of data broadcasting makes the detection of intrusion difficult in IoT. Therefore, an effective IDS is developed using the CRFC to classify the attack classes from normal traffic. At first, the maximum noise margin and min-max approach is used to preprocess the data. Then the preprocessed data is given as input to the CRFC where a feature importance-based cost matrix is included for precise classification. The developed CRFC is adapted to various attack types such as DoS, Probe, R2L and U2R by considering the data and attack distribution. This adaption is used for defining the CRFC's tree structures which provides the reliable intrusion detection.

Table 8
Comparison for NSL-KDD dataset.

Method	Accuracy (%)	F1-measure (%)
OCSVM [18]	99.8	99.4
CRFC-IDS	99.957	98.811

Table 9
Comparison for UNSW-NB15 dataset.

Method	Accuracy (%)	F1-measure (%)
OCSVM [18]	99.3	99.2
DBF [20]	99.41	NA
CRFC-IDS	99.953	99.311

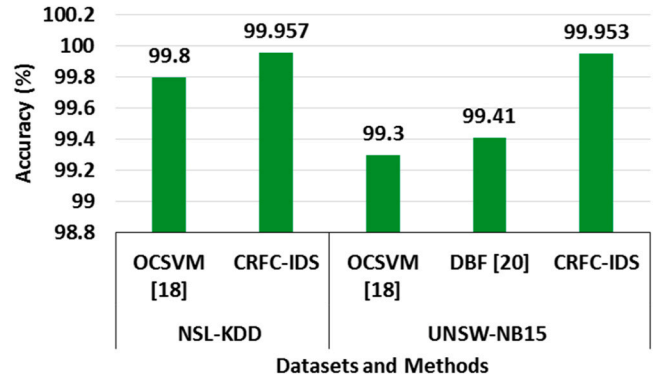


Fig. 6. Comparison of accuracy.

Moreover, the CRFC does not provide any random decision because of the improvisation of splitting criteria based on the cost matrix. Additionally, the Spark, Kafka and Scikit-learn Python libraries are used for a distributed environment and CRFC design for further improving the classification performances. From the results, it is concluded that the CRFC-IDS provides better performance than the OCSVM and DBF. The accuracy of CRFC-IDS for the NSL-KDD dataset is 99.957% which is highest, when compared to the OCSVM and DBF. As a further extension, the pruning and hyperparameter optimization can be incorporated when the IDS is processed in Big data with a variety of classes/ subcategories. In the future, deep learning approaches are fit to be used for improving the performance of IDS.

Funding

This research received no external funding.

Ethics approval

I/We declare that the work submitted for publication is original, previously unpublished in English or any other language(s), and not under consideration for publication elsewhere.

Consent to participate/informed consent

Not applicable.

Consent for publication

I certify that all the authors have approved the paper for release and are in agreement with its content.

CRediT authorship contribution statement

K. Pramilarani: Conceptualization, Implementation, Resource acquisition, Data collection, Writing – original draft, Report writing. **P. Vasanthi Kumari:** Project administration, Validation, Visualization, Formal analysis, Writing – review & editing. All authors approved the final manuscript.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The datasets generated and/or analysed during the current study are available in the [NSL-KDD] and [UNSW-NB15] repositories: NSL-KDD = <https://www.kaggle.com/datasets/hassan06/nslkdd?select=KDD-Train1.jpg>; UNSW-NB15 = <https://research.unsw.edu.au/projects/unsw-nb15-dataset>.

References

- [1] L. Nie, W. Sun, S. Wang, Z. Ning, J.J.P.C. Rodrigues, Y. Wu, S. Li, Intrusion detection in green internet of things: a deep deterministic policy gradient-based algorithm, *IEEE Trans. Green. Commun. Netw.* 5 (2021) 778–788, <https://doi.org/10.1109/TGCN.2021.3073714>.
- [2] Y. Kayode Saheed, A. Idris Abiodun, S. Misra, M. Kristiansen Holone, R. Colombo-Palacios, A machine learning-based intrusion detection for detecting internet of things network attacks, *Alex. Eng. J.* 61 (2022) 9395–9409, <https://doi.org/10.1016/j.aej.2022.02.063>.
- [3] M.H. Nasir, J. Arshad, M.M. Khan, Collaborative device-level botnet detection for internet of things, *Comput. Secur.* 129 (2023), 103172, <https://doi.org/10.1016/j.cose.2023.103172>.
- [4] Y. Wu, L. Nie, S. Wang, Z. Ning, S. Li, Intelligent intrusion detection for internet of things security: a deep convolutional generative adversarial network-enabled approach, *IEEE Internet Things J.* 10 (2023) 3094–3106, <https://doi.org/10.1109/JIOT.2021.3112159>.
- [5] V. Morfino, S. Rampone, Towards near-real-time intrusion detection for IoT devices using supervised learning and apache spark, *Electronics* 9 (2020) 444, <https://doi.org/10.3390/electronics9030444>.
- [6] M.B. Gorzalczy, F. Rudzinski, Intrusion detection in internet of things with MQTT protocol—an accurate and interpretable genetic-fuzzy rule-based solution, *IEEE Internet Things J.* 9 (2022) 24843–24855, <https://doi.org/10.1109/JIOT.2022.3194837>.
- [7] M.M. Alani, A.I. Awad, An intelligent two-layer intrusion detection system for the internet of things, *IEEE Trans. Ind. Inf.* 19 (2023) 683–692, <https://doi.org/10.1109/TII.2022.3192035>.
- [8] R. Zhao, G. Gui, Z. Xue, J. Yin, T. Ohtsuki, B. Adebisi, H. Gacanin, A novel intrusion detection method based on lightweight neural network for internet of things, *IEEE Internet Things J.* 9 (2022) 9960–9972, <https://doi.org/10.1109/JIOT.2021.3119055>.
- [9] D.C. Attota, V. Mothukuri, R.M. Parizi, S. Pouriyeh, An ensemble multi-view federated learning intrusion detection for IoT, *IEEE Access* 9 (2021) 117734–117745, <https://doi.org/10.1109/ACCESS.2021.3107337>.
- [10] M. Vishwakarma, N. Kesswani, DIDS: a deep neural network based real-time intrusion detection system for IoT, *Decis. Anal. J.* 5 (2022), 100142, <https://doi.org/10.1016/j.dajour.2022.100142>.
- [11] H. Lu, T. Wang, X. Xu, T. Wang, Cognitive memory-guided autoencoder for effective intrusion detection in internet of things, *IEEE Trans. Ind. Inf.* 18 (2022) 3358–3366, <https://doi.org/10.1109/TII.2021.3102637>.
- [12] M. Aslam, D. Ye, A. Tariq, M. Asad, M. Hanif, D. Ndzi, S.A. Chelloug, M.A. Elaziz, M.A. Al-Qaness, S.F. Jilani, Adaptive machine learning based distributed denial-of-services attacks detection and mitigation system for SDN-enabled IoT, *Sensors* 22 (7) (2022) 2697.
- [13] J. Chen, H. Huang, A.G. Cohn, D. Zhang, M. Zhou, Machine learning-based classification of rock discontinuity trace: SMOTE oversampling integrated with GBT ensemble learning, *Int. J. Min. Sci. Technol.* 32 (2) (2022) 309–322.
- [14] Z. Wang, J. Liu, L. Sun, EFS-DNN: an ensemble feature selection-based deep learning approach to network intrusion detection system, *Secur. Commun. Netw.* 2022 (2022), 2693948, <https://doi.org/10.1155/2022/2693948>.
- [15] O. Azeroual, A. Nikiforova, Apache spark and MLib-based intrusion detection system or how the big data technologies can secure the data, *Information* 13 (2022) 58, <https://doi.org/10.3390/info13020058>.
- [16] R. Alghamdi, M. Bellaiche, Evaluation and selection models for ensemble intrusion detection systems in IoT, *IoT* 3 (2022) 285–314, <https://doi.org/10.3390/iot3020017>.
- [17] A.A. Hagar, B.W. Gawali, Apache spark and deep learning models for high-performance network intrusion detection using CSE-CIC-IDS2018, *Comput. Intell. Neurosci.* 2022 (2022), 3131153, <https://doi.org/10.1155/2022/3131153>.
- [18] H. Alazzam, A. Sharieh, K.E. Sabri, A lightweight intelligent network intrusion detection system using OCSVM and Pigeon inspired optimizer, *Appl. Intell.* 52 (2022) 3527–3544, <https://doi.org/10.1007/s10489-021-02621-x>.
- [19] M. Sarhan, S. Layeghy, N. Moustafa, M. Gallagher, M. Portmann, Feature extraction for machine learning-based intrusion detection in IoT networks, *Digit. Commun. Netw.* (2022), <https://doi.org/10.1016/j.dcan.2022.08.012>.
- [20] O. Alkadi, N. Moustafa, B. Turnbull, K.-K.R. Choo, A deep blockchain framework-enabled collaborative intrusion detection for protecting IoT and cloud networks, *IEEE Internet Things J.* 8 (2021) 9463–9472, <https://doi.org/10.1109/JIOT.2020.2996590>.
- [21] A. Alferaidi, K. Yadav, Y. Alharbi, N. Razmjoo, W. Viriyasitvat, K. Gulati, S. Kautish, G. Dhiman, Distributed deep CNN-LSTM model for intrusion detection method in IoT-based vehicles, *Math. Probl. Eng.* 2022 (2022), 3424819, <https://doi.org/10.1155/2022/3424819>.
- [22] S. Baniassadi, O. Rostami, D. Martín, M. Kaveh, A novel deep supervised learning-based approach for intrusion detection in IoT systems, *Sensors* 22 (2022) 4459, <https://doi.org/10.3390/s22124459>.
- [23] M. Zeeshan, Q. Riaz, M.A. Bilal, M.K. Shahzad, H. Jabeen, S.A. Haider, A. Rahim, Protocol-based deep intrusion detection for DoS and DDoS attacks using UNSW-NB15 and Bot-IoT data-sets, *IEEE Access* 10 (2022) 2269–2283, <https://doi.org/10.1109/ACCESS.2021.3137201>.
- [24] B.E. Bouya-Moko, E.K. Boahen, C. Wang, Fuzzy local information and bhattacharya-based C-means clustering and optimized deep learning in spark framework for intrusion detection, *Electronics* 11 (2022) 1675, <https://doi.org/10.3390/electronics11111675>.
- [25] K.D. Devprasad, S. Ramanujam, S.B. Rajendran, Context adaptive ensemble classification mechanism with multi-criteria decision making for network intrusion detection, *Concurr. Comput. Pract. Exp.* 34 (21) (2022), e7110.
- [26] S. Mohamed, R. Ejali, Deep SARSA-based reinforcement learning approach for anomaly network intrusion detection system, *Int. J. Inf. Secur.* 22 (1) (2023) 235–247.
- [27] A.S. Alqahtani, FSO-LSTM IDS: hybrid optimized and ensembled deep-learning network-based intrusion detection system for smart networks, *J. Supercomput.* 78 (7) (2022) 9438–9455.
- [28] S. Wang, W. Xu, Y. Liu, Res-TranBiLSTM: an intelligent approach for intrusion detection in the Internet of Things, *Comput. Netw.* 235 (2023), 109982.
- [29] M. Tavallaee, E. Bagheri, W. Lu, A.A. Ghorbani, A detailed analysis of the KDD CUP 99 data set, in: 2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications, IEEE, Ottawa, ON, Canada, 2009, pp. 1–6, <https://doi.org/10.1109/CISDA.2009.5356528>.
- [30] N. Moustafa, J. Slay, UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set), in: 2015 Military Communications and Information Systems Conference (MilCIS), IEEE, Canberra, ACT, Australia, 2015, pp. 1–6, <https://doi.org/10.1109/MilCIS.2015.7348942>.
- [31] S.J. Stolfo, Wei Fan, Wenke Lee, A. Prodromidis, P.K. Chan, Cost-based modeling for fraud and intrusion detection: results from the JAM project, DISCEX'00. Proceedings DARPA Information Survivability Conference and Exposition, IEEE Comput. Soc., Hilton Head, SC, USA, 2000, pp. 130–144, <https://doi.org/10.1109/DISCEX.2000.821515>. DISCEX'00.
- [32] X. Larriva-Novo, V.A. Villagrà, M. Vega-Barbas, D. Rivera, M. Sanz Rodrigo, An IoT-focused intrusion detection system approach based on preprocessing characterization for cybersecurity datasets, *Sensors* 21 (2021) 656, <https://doi.org/10.3390/s21020656>.
- [33] F. Ouyang, B. Guo, L. Ouyang, Z. Liu, S. Lin, W. Meng, X. Huang, H. Chen, H. Qiu-gen, S. Yang, Comparison between linear and nonlinear machine-learning algorithms for the classification of thyroid nodules, *Eur. J. Radiol.* 113 (2019) 251–257, <https://doi.org/10.1016/j.ejrad.2019.02.029>.
- [34] D. Chumachenko, I. Meniaïlov, K. Bazilevych, T. Chumachenko, S. Yakovlev, Investigation of statistical machine learning models for COVID-19 epidemic process simulation: random forest, K-nearest neighbors, gradient boosting, *Computation* 10 (2022) 86, <https://doi.org/10.3390/computation10060086>.
- [35] D. Marković, D. Vujčić, S. Tanasković, B. Đorđević, S. Randić, Z. Stamenković, Prediction of pest insect appearance using sensors and machine learning, *Sensors* 21 (2021) 4846, <https://doi.org/10.3390/s21144846>.